

1 写在最前

这个文档会记录一些我觉得比较合理的学习内容，但可能会有遗漏或者不合理的地方，如果有任何意见/勘误，欢迎提出。

大体包括以下几个方面（后面想到再加）：

1. 深入了解 Transformer 架构，里面涉及几个重要的点：Attention、(Gated) FFN、LayerNorm、Position Embedding、Tokenizer（以及分词算法比如 BPE）、通常说的两阶段训练（Pre-Train 和 Post-Train）、训练目标、自回归解码过程。
2. 几种常见的后训练：SFT、PO（DPO）、RL（PPO、GRPO），可以短期内不细究实现和公式，先认识一下几种方法各自的优缺点和思路，建立图像。
3. 一些 PEFT 方法，基本上特指 LoRA，建议深入学习，结论和效果都是很漂亮的。然后可以大概了解一下量化模型的做法。
4. 一些分布式训练的技术，主要看看 DeepSpeed 的几个 ZeRO 和它们的通信过程，了解特点、建立图像就可以，短期内不需要细究实现。
5. 一些推理加速的技术，基本上特指 vLLM，大概了解原理就可以，短期内不需要细究实现。
6. 完整的现代模型训练管线，读几篇技术报告。了解一个模型从预训练到后训练、评测、部署的大致流程。

推荐的学习路线：

1. 先通过博客之类，建立一下概念地图。
2. 看看 Transformer 架构和代码实现，熟悉一下 MHA（多头自注意力）。
3. 跑一遍模型推理（用 vLLM 的 openai compatible server），以及评测（用开源评测框架，随便测试什么它已经支持的 benchmark），加分项是读一下评测框架的核心代码。这个过程中比较重要的是理解 chat template 和推理参数。
4. 再看看 SFT 和 LoRA 的原理和代码实现（huggingface 的 transformers 库）。
5. 再看看 PPO、DPO、GRPO 各自的原理，可以先不看源码，重点看一下训练数据长什么样。
6. 看看 LlamaFactory 和 veRL 的文档，自己跑一下 SFT 和 GRPO。LlamaFactory 的源码是封装的 trl 和 transformers，可以看看数据组织、模板和参数之类的；veRL 的比较复杂，这个阶段可以不看。
7. 读一些技术报告，Llama3、DeepSeek-R1、Smol playbook
8. 倒回去研究一下之前的 SFT、GRPO 训练过程，

注：上面写的东西太多太杂了，两周基本上不太可能学得完，所以我会列一下优先级（必学、现阶段了解就行、了解就行）。以及推荐的学习交付形式（不强求）。

学习过程中不可避免的涉及很多 github 项目或者官方文档，官方文档通常是仅次于源码实现的准确信息源，推荐养成读官方文档的熟练度。在读文档不知从哪里下手的时候，通常可以通过类似“Introduction”的章节了解项目的大概介绍，从类似“Installation”的章节了解项目的几种推荐安装方式，从类似“QuickStart”的章节了解如何快速使用项目最核心的功能。

整个学习过程如果会整理成文档、代码等交付物的话，推荐一开始用 git 做管理，顺便熟悉一下 git 操作。

2 概念/技术地图

这个阶段的目标不是掌握所有细节，而是先建立一张大语言模型技术地图：知道 Transformer、预训练模型、GPT/BERT、现代 decoder-only LLM、后训练分别处在什么位置，以及它们大概解决什么问题。

首先是 Transformer 和预训练模型（包括 BERT、GPT 等等），了解一下历史（近年做法会有一些区别）。

【必学】在此之前，假设你了解 Transformer，如果了解可以跳过这一段或者速通一下三个资料查漏补缺。如果不了解的话，可以读一下博客 [The Illustrated Transformer](#)；Transformer 的代码实现可以参考 [The Annotated Transformer](#)。原论文作为参考 [Attention is all you need](#)。

【了解】了解一些预训练模型和自然语言处理（NLP）的基本概念：可以首先看这篇博客：[《从 Word Embedding 到 Bert 模型—自然语言处理中的预训练技术发展史》](#)，这引入了 GPT 和 BERT。再看看这篇博客：[《乘风破浪的 PTM：两年来预训练模型的技术进展》](#)。没必要细究，现阶段这些都属于上古技术了，短期可以带着涨见识的心态速通一下。

【现阶段了解】真正的重头戏是 LLM101，但是两周肯定啃不完，慢慢看吧：[LLM101-1-overview](#)。现阶段看看里面的几张架构图和下面这张表就行，之后可能会经常回来看。

Component	Technologies	Chapters
Model Architecture	Transformers, Attention, LayerNorm, Residual Connections	4, 5
Tokenization	Byte Pair Encoding (BPE), minBPE	6
Optimization	AdamW, Initialization Strategies	7
Performance	CPU/GPU Optimization, Mixed Precision (fp16, bf16, fp8)	8, 9
Scaling	Distributed Data Parallel (DDP), ZeRO	10
Inference	KV-Cache, Quantization	12, 13
Adaptation	SFT, PEFT, LoRA, RLHF, PPO, DPO	14, 15
Deployment	API, Web Application	16
Extensions	VQVAE, Diffusion Transformer	17

2.1 整理一下学习内容和一些细节

- 1. [《从 Word Embedding 到 Bert 模型—自然语言处理中的预训练技术发展史》](#)：目标不是记模型细节，而是画出发展线：word2vec -> ELMo -> GPT -> BERT -> GPT-style LLM
- 2. [《乘风破浪的 PTM：两年来预训练模型的技术进展》](#)：重点在模型范式分类，不需要记 ALBERT、XLNet、RoBERTa 的细节，只要知道这些是围绕数据、目标、结构、训练策略做改进。
- 3. [LLM101-1-overview](#)：对 LLM 的技术栈分层

不要求推公式、背模型细节、复现代码。

2.2 交付

类似技术地图的东西，形式随意，文档、思维导图、PPT、其他。主要是谈谈你对整个概念的图像，以及其中不太理解、不会区分的东西。也可以细节聊聊对其中一些概念的认识。建议控制在 1-3 页，不需要写成长报告。

GPT 推荐的交付内容，可以不这么做：

- 1. 一张你理解的 LLM 技术地图。

2. 10-20 个关键词的简短解释，例如 pretrain、finetune、SFT、RLHF、DPO、tokenizer、causal LM、masked LM、attention、LoRA。
3. 3-5 个你目前还不理解、容易混淆、或者想继续追问的问题。（我觉得更多也行）
4. 简单说说你觉得自己更感兴趣的方向：模型结构、数据、训练、评测、推理、源码，或者特别的：RL。

这个阶段可以思考以下几个问题：

1. 什么是预训练？预训练和微调的区别？
2. GPT 和 BERT 的预训练目标有什么差异？
3. 随便选一个类 Transformer 模型，讲讲它的以下内容：Attention、(Gated) FFN、LayerNorm、Position Embedding、Tokenizer、训练目标。
4. 比较开放的思考：为什么现在大语言模型主流是 decoder-only causal LM？
5. Chat model 和 base model 有什么区别？为什么预训练之后还需要后训练？

3 模型推理和评测

这个阶段是实操，目标是理解自回归解码。所以实际上需要看代码实现的只有 GPT 架构模型，后面的 transformers、vLLM、openai-sdk 等等都是只要照着文档用起来就行，不需要读源码。

因为下面写的这些东西跑起来很容易，所以这个阶段不希望你靠别人总结过的教程博客、AI 等方式直接跑起来，更希望你读英文文档自己理解使用方式。锻炼一下读文档的能力。所以，下面的实验都需要你安装最新版本的库（因此网上很多教程应该会遇到 bug）。

重复一遍：

下面不可避免的涉及很多 github 项目或者官方文档，官方文档通常是仅次于源码实现的准确信息源，推荐养成读官方文档的熟练度。在读文档不知从哪里下手的时候，通常可以通过类似“Introduction”的章节了解项目的大概介绍，从类似“Installation”的章节了解项目的几种推荐安装方式，从类似“QuickStart”的章节了解如何快速使用项目最核心的功能。

【必做】找一个简单一点的 GPT 架构的模型，上面的 Annotated Transformer 也行，重点看看 MHA 的输入输出、attention mask、自回归预测。

【必做】选一个模型，暂定 [meta-llama/Llama-3.2-1B-Instruct](#) 吧，有服务器权限之后细说，用 [transformers](#) 跑一遍模型推理。固定一组 prompts，尝试不同 temperature、top_p、max_new_tokens，记录输出。

这个过程中，重点关注一下 chat template：你输入的 messages 实际上是以什么样被模型实际看到的。

【必做】使用 [vLLM](#) 部署 OpenAI-Compatible server，然后用 openai-sdk 请求这个 server 获取响应。同样地，固定一组 prompts，尝试不同 temperature、top_p、max_new_tokens，记录输出。不需要深究 vLLM 的实现原理和源码，跑起来就行。

【必做】使用一个开源评测框架跑一个它已经支持的 benchmark。不要求读源码，先理解模型输出如何被解析、分数如何计算。

3.1 整理一下学习内容和一些细节

1. 读一下 GPT 架构模型的实现：MHA、Attention Mask、自回归预测
2. 用 transformers 库实现推理，在上面给出的文档链接里，目录是这样：

Transformers Docs =	{	Installation
		* Quickstart * ← 重点看
		BASE CLASSES
		MODELS
		PREPROCESSORS
		INFERENCE
		PIPELINE API
		* GENERATE API * ← 重点看
		Text generation
		Decoding methods
		Generation features
		Prompt engineering
		Perplexity of fixed-length models
		OPTIMIZATION
		* CHAT WITH MODELS * ← 重点看
		Chat basics
		Chat templates
		Chat message patterns
		Multimodal chat templates
		Tool use
...		

1. 用 vLLM 部署 [OpenAI-Compatible server](#), 并且用 [openai-sdk](#) 调用。调用方式可以看 [github 仓库](#) 或者 [pypi](#)。
2. 用开源测评框架 (比如 [evalscope](#)) 跑一下测评, 看看结果文件和几个指标。

3.2 交付

实验的交付会更加随意, 代码+运行结果就可以。只有代码也行。